



Government College University Faisalabad
Government Graduate College Satiana Road, Faisalabad
Department of Computer Science
(PROJECT PROPOSAL)



	Day	Month	Year
DATE	03-10-2026		

Project Code :

Project Title : AgriPulse Analytics Soil-Test Driven Farm & Financial Decision Engine

Problem Statement:

A farmer in Punjab must decide what to sow, how much to spend, and how much fertiliser to buy before the season starts. Almost every tool that exists today asks him for answers he does not have, or needs a signal his village does not get.

Pakistani farming decisions are made under three conditions at once: the farmer has very little money, the decision is made once and cannot be undone until harvest, and the information needed to make the decision is locked inside government offices and paid services.

We studied 13 existing tools 5 working in Pakistan (Bakhabar Kissan, Kissan, Agrixia, MandiOye, PAR) and 8 international ones (PlantVillage/Nuru, Leafwise, LeafAid AI, Kisan Sarathi, Kisan Suvidha, Miraitu, KisanPe, CropKhata/Farm Manager). **We found three problems that this project can actually solve.**

- Farming decisions are made without real information. The soil test has already been done the Punjab Agriculture Department runs 36 soil testing laboratories, eight divisional and twenty-eight district, and each one issues the farmer a written report of his nitrogen, phosphorus, potassium and pH. That report is a page of numbers, and nobody turns it into a crop choice, a fertiliser quantity or a cost sheet so the crop is chosen blind and money and water are wasted on the wrong dose.
- There is no planning before the harvest. On the day of sowing the farmer does not know his total expense, his water requirement, his expected yield or his profit. He plans in his head from last year's prices, cannot compare two crops side by side, and finds out the cost only when the season ends.
- Existing tools depend on hardware and paid services. They need physical weather stations, a working internet connection, a monthly data subscription, or a trained model on a server so in the field the farmer either pays or goes blind.

Aim and Objective:

The aim of this project is as follows:

Tools to be used:

- React 18
- Vite
- Tailwind CSS
- Recharts
- React Router
- Fetch API client
- Node.js
- Express.js
- MongoDB
- MongoDB Compass
- JWT
- Bcrypt
- Zod
- Cors
- Dotenv
- Morgan
- Nodemon
- Git + GitHub

- **Turn one soil test report into a complete sowing decision.** The farmer enters the nitrogen, phosphorus, potassium and pH figures printed on the report issued to him by his district soil laboratory, together with his land size, and gets back the four things he must decide before sowing total cost, water need, fertiliser bags and expected profit instead of guessing from memory.
- **Use published government and FAO methods, not invented formulas.** Every figure the system produces must match a named public publication, so the supervisor and the farmer can both check it.
- **Run the whole thing on one laptop.** Hardware, a network connection and a monthly fee are the real barriers in the field, so the application is built to run on a single machine with nothing else attached.
- **Show the source of every number on the screen.** A figure nobody can trace is worth nothing to the farmer and nothing to the evaluator, so each citation sits next to the value it belongs to.
- **Teach one idea, not just hand out numbers.** The farmer should leave understanding that cultivating more land with the same budget raises total profit but lowers the return on every rupee he spends.

The objectives of this project are as follows:

- **Read the soil test report and show what the soil is short of.** A radar chart will plot the farmer's nitrogen, phosphorus, potassium and pH against the target range for his soil type, so the deficiency is visible instead of printed.
- **Calculate the exact number of Urea, DAP and MOP bags the field needs.** The calculation uses the Government of Punjab SFRI Guide-V formula and is validated against the published PCPA cotton figure of 2.35 bags Urea and 1.04 bags DAP per acre.
- **Calculate the full cost of sowing, line by line, in Pakistani Rupees.** Land preparation, seed, fertiliser, labour, water and transport are shown separately, with the PCPA Cotton Book or PAU Package of Practices figure named beside each line.
- **Calculate the water requirement and the money the crop will return.** Total water in litres and the number of irrigations will come from the FAO-56 method; expected yield, net profit, return on investment and break-even price will come from published district yield bands and official prices.
- **Let the farmer compare options before he spends money.** Sliders for area, budget and crop recompute cost, profit and return immediately inside the browser, with no network call and nothing written to the database.
- **Keep the farmer's data private and the app usable with no signal.** Login is protected by a JWT token and every query is filtered to the logged-in farmer, while every screen and every calculation still works with the router off.

Scope of the Project:

This project will deliver a five-page web application built from four modules login and registration, land and climate profiling, resource and financial planning, and NPK fertiliser advising plus a what-if simulator that compares two plans side by side before money is committed. The farmer will register once, record his district, land area and soil type, enter the nitrogen, phosphorus and potassium values from his soil test report, and receive the Urea, DAP and MOP bag counts, a full cost breakdown in Rupees, the water requirement in litres, the expected yield, the net profit, the return on investment and the break-even price, with the name of the public source printed beside every figure. The application will run on a single laptop, hold no machine-learning model of any kind, and load every asset from the local server rather than the internet.

- **What is being built.** Five pages across four modules login and register, land and climate profiler, resource and financial planner, NPK soil fertiliser adviser plus a what-if simulator that compares two plans side by side on the same screen.

- The rule that governs the data.** A crop appears in the dropdown only if its cost and yield figures have a named, downloadable public source. Where a crop has no sourced figures it is shown as unavailable rather than filled in with a guess. This is why the crop count is written as a target and not a promise.
- What this project does not attempt.** Leaf disease identification from a photo, live mandi price feeds, live weather and IoT sensors are outside what a single-laptop project can do honestly. Each needs a trained model, a paid server or a second codebase, and none is needed to solve the three problems above. Saved plans, PDF reports, Urdu voice output, a native mobile app and an admin panel are left out for the same reason they are not required by the three problems.
- Coverage of the three problems.** Of the 13 factors inside the three problems, 11 will be fully addressed and 2 partly addressed. The scorecard below shows the split.
- Seven weeks of delivery, with a checkable result each week.** Week 5 is reserved entirely for the one test that decides the project: the calculated bag counts must match a published figure. Every other week ends with a stated result that can be seen.
- How it will be verified.** Unit checks on the four calculation services including the published-figure check, plus a run with the router physically switched off.

Functional Requirements	Non-Functional Requirements
1 A visitor can register with full name, email and password, and a registered farmer can log in and reach the protected pages. 2 The system will keep the password only as a scrambled hash and never in plain text, and will refuse any protected request that arrives without a valid login token. 3 One farmer can never read or change another farmer's land record or plan. 4 The farmer selects his district from a list of Punjab and Sindh districts and sees the climate card for that district and the current month. 5 The farmer enters his land area in acres, from 0.1 to 100, and selects one soil type; the details are saved against his own account. 6 The farmer enters the nitrogen, phosphorus and potassium values from his soil test report, together with the pH reading, and the system converts units when the report states them in parts per million rather than kilograms per hectare, using the standard soil depth and bulk-density conversion. 7 The system will rank each nutrient as low, medium or adequate against the target range for that soil type, and show the result as a radar chart of nitrogen, phosphorus, potassium and pH. 8 The system will convert the nutrient still needed into whole bags of Urea, DAP and	1 The application will work with the network completely off: every screen opens and every calculation runs, including after a page refresh. 2 Zero runtime network dependencies no external API, CDN, font host, analytics or telemetry is contacted while the app is in use. All reference data is bundled with the application. 3 A page will appear in under two seconds on a mid-range laptop, measured on the demo machine and shown on screen during the demonstration. 4 Any single calculation will finish in under 300 milliseconds, measured by the unit checks. 5 Moving a simulator slider will recompute in under 50 milliseconds with no visible lag. 6 Passwords will be stored only as bcrypt hashes, every protected endpoint will require a valid token, and no user can read or modify another user's data. 7 All secrets will come from environment variables and never from source code, and the environment file will be excluded from version control. 8 Every calculated number will be traceable to a named public source, and each API response will carry the source of the

MOP, rounding up, and multiply by the farmer's acres to give total bags including returning zero bags when a nutrient is already above target. 9 The system will show the fertiliser cost in Rupees and a "How this was calculated" panel that names the SFRI Punjab Guide-V method as the source. 10 The system will return a cost breakdown with named lines land preparation, seed, fertiliser, labour, water and transport each with its Rupee amount and its source shown next to it. 11 The system will show total water in litres, the number of irrigations, the expected yield in maund, the net profit, the return on investment and the break-even price, and will warn when the cost exceeds the entered budget. 12 The what-if simulator will let the farmer move sliders for area, budget and crop, show the baseline against the scenario with the change as a percentage, update every figure live, reset to the baseline on one click, and state on screen that results are session-only.	figures it returns. 9 The four calculation services will be covered by unit checks including the published-figure check and the test run will be green. 10 Invalid input will be rejected with a clear message and will never crash the server; malformed requests will return a client error rather than a server error. 11 A farmer who has never seen the application will complete "set up land see fertiliser bags" in under three minutes, every control will be reachable by keyboard with a visible focus ring, no screen will scroll sideways on a small phone, and colour will never be the only signal on screen. 12 No farmer data will leave the machine, the application will run on Windows, Linux and macOS from a fresh clone, and every module will be documented in the four guidance documents.
---	--

Particulars of the students:

Sr. #	Registration	Name in Full	Email	Contact #	CGPA	Signatures
1	2023-GCUF-074974	Abdullah Munir	abdullahmunir0326@gmail.com	03284532486		
2	2023-GCUF-074992	Muhammad Hamza	Hamzeyjutt@gmail.com	03407430058		

Remarks:	Signatures and Date
Approved: ☐ YES ☐ NO	Coordinator Head Of Department